

BIL401

PROJECT STATUS REPORT

Beril Aydın 231101002

Kaan Baykal 221101028

CONTENTS

1. DATA COLLECTION STATUS.....	3
1.1 Dataset Download and Access.....	3
1.2 Data Analysis.....	3
1.3 Timestamp Injection Prototype.....	4
2. PLATFORM AND SYSTEMS STATUS.....	4
2.1 Docker Compose Environment.....	4
2.2 Apache Kafka.....	5
2.3 Apache DataFusion.....	5
2.4 Apache Cassandra.....	6
2.5 End-to-End Connectivity Test (End-to-End Pipeline).....	6
3. INSTALLATION PROBLEMS, KNOW-HOW, DEMO RUNS.....	7
3.1 Problems Encountered and Their Solutions.....	7
3.2 Know-how Gained.....	7
3.3 Demo Run Evidence.....	7
4. PAPER DRAFT.....	8
Abstract.....	8
Related Work.....	8
Proposed Implementation.....	9
5. NEXT STEPS.....	10
6. REFERENCES.....	11

1. DATA COLLECTION STATUS

1.1 Dataset Download and Access

The CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System) dataset, which will be used in the project, was downloaded via the Kaggle website (www.kaggle.com/datasets/chethuhn/network-intrusion-dataset). Obtained in static CSV format, the raw dataset is approximately 880 MB in size and contains roughly 2.8 million network flow records.

The CICIDS2017 dataset consists of eight separate CSV files covering normal network traffic and various attack scenarios from eight different days. Since our primary goal in this project is to simulate a high-volume and continuous real-time network traffic stream, a consolidated data structure containing all of the approximately 2.8 million records will be used instead of performing day-based filtering. Thanks to this approach, a long-term and continuous synthetic load can be generated on the data processing pipeline (consisting of Apache Kafka, Apache DataFusion, and Apache Cassandra components) allowing for a more comprehensive evaluation of the system's performance under high data volume.

Furthermore, since the data in the CSV files consists of pre-processed aggregate network flow features, it does not directly represent a real time data stream. Therefore, an adaptation will be applied to the static data in order to evaluate the system's real-time stream processing capabilities. To properly calculate sliding and tumbling time windows on Apache Kafka and Apache DataFusion, synthetic timestamps (injected_timestamp) will be generated for each record during data ingestion into the pipeline and appended to the data stream. In this way, the static dataset can be fed into the system in a manner that mimics real-time data stream behavior.

1.2 Data Analysis

An exploratory analysis was performed on the eight days of CSV files to verify the general structure and quality of the dataset to be used in the project architecture. The results obtained are as follows:

- **Total Row/Column Count Verification:** When the eight separate CSV files making up the dataset were examined, a total of 2,830,743 rows of data were reached, which aligns with the project's target expectation of "approximately 2.8 million records." Additionally, it was verified that the dataset consists of 79 columns, including features and the label.
- **Missing Value / Corrupted Row Rate and Column Detection:** In the analysis conducted to measure the quality of the dataset, missing, erroneous, or infinite values were detected in only 2,867 rows in total. Compared to the overall data volume, the corrupted row rate is at a remarkably low level of %0.1013. Detailed column analysis revealed that all of these missing/erroneous values were concentrated in the Flow Byte's and Flow Packets's columns. This situation highlighted the necessity of a data-cleaning step prior to ingestion: rows containing missing or infinite values in any column — predominantly concentrated in the Flow Bytes/s and Flow Packets/s columns — are detected and removed entirely from the dataset during the preparation stage, rather than being repaired or imputed.
- **Destination Port and Load Distribution Analysis:** The "Destination Port" feature, which is planned to be used as the Apache Kafka partitioning key in the project architecture, was specifically examined. According to the distribution results, it was observed that 33.84% of

the total network traffic flowed through Port 53, 21.86% through Port 80, and 17.86% through Port 443. This resulting data skew was accepted as the fundamental reference point for designing load-balancing strategies to prevent potential "hot partition" (overloaded partition) bottlenecks that could occur on Kafka.

- **Label Distribution:** When the class distribution in the dataset was examined, it was observed that the vast majority of the traffic consisted of normal flows. 80.30% of the total data (2,273,097 records) was labeled as BENIGN (normal). The remaining portion of approximately 19.70% contains various attack types. The major attack types were identified as DoS Hulk (8.16%), PortScan (5.61%), and DDoS (4.52%), respectively. On the other hand, attack types such as Infiltration (0.0013%), Sql Injection (0.0007%), and Heartbleed (0.0004%) are observed very rarely within the dataset. The class distribution shows a clear imbalance between normal traffic and attack traffic. This situation will allow the rule-based filtering mechanism to be developed to be tested under different attack intensities.

1.3 Timestamp Injection Prototype

To simulate a real-time network traffic stream and perform sliding and tumbling time window calculations on Apache DataFusion, a Python prototype script was developed to append synthetic timestamps to the static CSV data.

- **Timestamp Generation:** In addition to the pre-processed flow features in the original dataset, the system's instantaneous time in milliseconds (Unix Epoch ms) was calculated as each flow record was read, and integrated into the data as a new column named `injected_timestamp`.
- **Prototype Status and Data Integrity:** The developed prototype was successfully executed and tested on the 8-day dataset (2,830,743 rows). It was verified that data integrity was maintained and the assigned timestamp format (Epoch) complied with the standards accepted by the data processing pipeline.
- **Performance and Live Streaming Throughput:** To assess the raw cost of timestamp injection in isolation, a synthetic benchmark assigned incrementing timestamps to 2,830,743 rows without I/O or serialization overhead, completing in 0.5418 seconds ($\approx 5,224,784$ rows/sec). This isolates the timestamp-assignment operation itself; it does not represent the throughput of the production producer, which additionally performs per-row datetime formatting, JSON serialization, and Kafka delivery, and is separately rate-limited via the `--rate` parameter (see Section 2.5).

2. PLATFORM AND SYSTEMS STATUS

The installation and testing of the four components (Kafka, DataFusion, Cassandra, Docker Compose) to be integrated under Systems Track have been successfully completed, and they have been made operational in an isolated local environment.

2.1 Docker Compose Environment

The system infrastructure is configured as a container architecture via Docker Compose. A `docker compose ps` check confirms that all three defined services (Zookeeper, Kafka, and Cassandra) are running in a healthy state:

```
berilay@BerilsHuawei: /intrusion-detection-pipeline$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
cassandra            cassandra:5.0                      "docker-entrypoint.s..."           cassandra  15 minutes ago Up 15 minutes 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp
kafka                confluentinc/cp-kafka:7.6.0        "/etc/confluent/dock..."           kafka      15 minutes ago Up 15 minutes 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
zookeeper            confluentinc/cp-zookeeper:7.6.0    "/etc/confluent/dock..."           zookeeper  15 minutes ago Up 15 minutes 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
```

The network topology between components is isolated on Docker's default bridge network, with only the necessary ports exposed to the host.

2.2 Apache Kafka

The Kafka and Zookeeper broker setup was completed using the confluentinc/cp-kafka:7.6.0 and confluentinc/cp-zookeeper:7.6.0 images. The network-traffic topic, which will carry the simulated traffic stream, has been created:

```
berilay@BerilsHuawei: /intrusion-detection-pipeline$ docker exec -it kafka kafka-topics --create --topic network-traffic
--bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
Created topic network-traffic.
berilay@BerilsHuawei: /intrusion-detection-pipeline$ docker exec -it kafka kafka-topics --describe --topic network-traffic
--bootstrap-server localhost:9092
Topic: network-traffic TopicId: wT1EbIyjRL2BrvZdsMMZrA PartitionCount: 3      ReplicationFactor: 1      Configs:
Topic: network-traffic Partition: 0 Leader: 1 Replicas: 1 Isr: 1
Topic: network-traffic Partition: 1 Leader: 1 Replicas: 1 Isr: 1
Topic: network-traffic Partition: 2 Leader: 1 Replicas: 1 Isr: 1
```

The Destination Port field is used as the partition key, ensuring that records are distributed across the three partitions as planned in the proposal.

2.3 Apache DataFusion

Apache DataFusion (version 54.0.0, via the datafusion-python binding) has been installed and integrated. In initial tests against static CSV data, SQL-based grouping (flow count by Destination Port) was validated first. Event-time tumbling-window aggregation was then prototyped separately on a synthetic timestamp column and, following successful validation, integrated directly into the Kafka consumer: each micro-batch's Injected_Timestamp field is truncated to a 1-second boundary via date_trunc, and window_start/window_end are derived from this event-time value rather than the wall-clock time at which the batch happens to be processed.

`python3 datafusion/test_query.py:`

```
DataFrame()
+-----+-----+
| Destination Port | flow_count |
+-----+-----+
| 53                | 90277      |
| 443               | 39228      |
| 80                | 20615      |
| 123               | 3578       |
| 8080              | 1261       |
| 22                | 1004       |
| 137               | 756        |
| 389               | 736        |
| 88                | 716        |
| 21                | 391        |
+-----+-----+
```

Since DataFusion does not provide native Kafka streaming ingestion support, a micro-batching approach was adopted for stream consumption (see Section 3.1).

2.4 Apache Cassandra

An Apache Cassandra 5.0 node has been set up with CQL (Cassandra Query Language) access. As part of the distributed storage strategy, a keyspace named `intrusion_detection` (SimpleStrategy, replication_factor=1) has been created. Two core table schemas are defined under this keyspace:

- **flagged_anomalies:** stores malicious network traffic that fails rule-based filtering (i.e., non-BENIGN flows). `dst_port` is used as the partition key, with `event_time` and `detection_id` as clustering keys.
- **windowed_traffic_metrics:** holds the periodic stream aggregations computed by DataFusion (total flows, benign flows, attack count, average duration).

2.5 End-to-End Connectivity Test (End-to-End Pipeline)

An end-to-end data flow scenario was tested to confirm that all components operate together correctly, using the following commands:

```
python3 producer/produce.py --file
```

```
data/clean/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv --rate 100 --limit 3000
```

```
berilay@BerilsHuawei: ~/intrusion-detection-pipeline$ python3 producer/produce.py --file data/clean/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv --rate 100 --limit 3000
... 500 messages sent
... 1,000 messages sent
... 1,500 messages sent
... 2,000 messages sent
... 2,500 messages sent
... 3,000 messages sent
Done. Total sent: 3,000 messages
```

```
python3 datafusion/consume_process.py --batch-size 100 --max-batches 30
```

```
berilay@BerilsHuawei: ~/intrusion-detection-pipeline$ python3 datafusion/consume_process.py --batch-size 100 --max-batches 30
30
Consumer started, waiting for messages...
Batch processed: 100 msgs, 0 anomalies, 23 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 39 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 26 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 19 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 21 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 5 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 5 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 4 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 7 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 4 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 21 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 18 window/port groups written to Cassandra
Batch processed: 100 msgs, 4 anomalies, 27 window/port groups written to Cassandra
Batch processed: 100 msgs, 3 anomalies, 43 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 56 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 20 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 16 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 26 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 39 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 20 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 14 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 9 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 5 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 10 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 21 window/port groups written to Cassandra
Batch processed: 100 msgs, 0 anomalies, 23 window/port groups written to Cassandra
Done. Total batches: 30
```

3000 records were sent to Kafka at a rate of 100 messages/second; `consume_process.py` then processed these records in micro-batches of 100 within a DataFusion context, computed 1-second tumbling windows per destination port based on each record's `Injected_Timestamp`, and persisted the results to the Cassandra database.

3. INSTALLATION PROBLEMS, KNOW-HOW, DEMO RUNS

3.1 Problems Encountered and Their Solutions

- **WSL2 memory limits:** Loading full CICIDS2017 CSV files (up to ~700K rows) into pandas caused "Cannot allocate memory" errors under WSL2's default memory cap (half of host RAM). Resolved by explicitly raising the WSL2 memory limit via `.wslconfig` and rewriting the data-preparation scripts to process files in 50,000-row chunks instead of loading them fully into memory.
- **Kafka/Zookeeper crash loop after restart:** After restarting the Docker Compose stack, Kafka repeatedly failed to start with `org.apache.zookeeper.KeeperException$NodeExistsException`, because Zookeeper retained a stale ephemeral broker registration from the previous session. Resolved by fully tearing down containers and volumes (`docker compose down -v`) and restarting Zookeeper before Kafka to guarantee a clean broker registration.
- **Duplicate column name in raw data:** The CICIDS2017 CSV header contains the column "Fwd Header Length" twice. Pandas tolerates this, but DataFusion's schema inference collapsed the duplicate, causing a field-count mismatch ("expected 78, got 79") when the raw CSV was read directly. Resolved by deduplicating column names (renaming the second occurrence to `Fwd Header Length_1`) during the cleaning step.
- **Label encoding corruption:** The "Web Attack" label values were corrupted (mojibake) when read with the wrong text encoding, because the source CSVs are encoded in cp1252 rather than UTF-8/latin1. Resolved by reading with the correct encoding and applying a regex-based normalization pass to the Label column.
- **No native Kafka connector in DataFusion:** Unlike Spark Structured Streaming, DataFusion has no built-in Kafka source connector. This was addressed with a custom micro-batch bridge — a Python consumer (`confluent-kafka`) that accumulates messages into fixed-size batches and loads each batch into a DataFusion `SessionContext` as a `DataFrame` — rather than a blocking issue.
- **GitHub HTTPS authentication failures:** Pushing to GitHub failed despite entering correct account credentials, because GitHub no longer accepts account passwords for HTTPS Git operations. Resolved by authenticating with a Personal Access Token instead.

3.2 Know-how Gained

- DataFusion's in-memory data processing speed is quite high; the main bottleneck instead occurs during row-by-row JSON deserialization when reading messages from Kafka.
- CICIDS2017 CSVs must be read with `encoding='cp1252'` to avoid label corruption; `latin1` alone does not fully resolve the issue.
- Chunked pandas processing (the `chunksize` parameter) is essential for reliably handling files of this size under constrained local resources.
- Keeping the working repository on the native Linux filesystem inside WSL2, rather than a Windows-side cloud-synced folder, meaningfully improves both I/O performance and Docker reliability.

3.3 Demo Run Evidence

Following the end-to-end test scenario described in Section 2.5, a verification query run against the Cassandra database confirms that the pipeline now computes distinct tumbling windows per port,

derived from each record's event time and executes successfully:

Windowed traffic metrics with attack counts:

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it cassandra cqlsh -e "SELECT dst_port, window_start, window_end, total_flows, benign_count, attack_count FROM intrusion_detection.windowed_traffic_metrics WHERE attack_count > 0 LIMIT 10 ALLOW FILTERING;"
```

dst_port	window_start	window_end	total_flows	benign_count	attack_count
80	2026-07-13 18:24:25.000000+0000	2026-07-13 18:24:26.000000+0000	12	10	2
80	2026-07-13 18:19:56.000000+0000	2026-07-13 18:19:57.000000+0000	16	13	3
80	2026-07-13 18:05:13.000000+0000	2026-07-13 18:05:14.000000+0000	9	8	1

(3 rows)

Anomaly records:

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it cassandra cqlsh -e "SELECT dst_port, event_time, label FROM intrusion_detection.flagged_anomalies LIMIT 10;"
```

dst_port	event_time	label
80	2026-07-13 18:24:26.014000+0000	PortScan
80	2026-07-13 18:24:25.952000+0000	PortScan
80	2026-07-13 18:24:25.901000+0000	PortScan
80	2026-07-13 18:24:25.580000+0000	PortScan
80	2026-07-13 18:24:25.457000+0000	PortScan
80	2026-07-13 18:24:25.447000+0000	PortScan
80	2026-07-13 18:24:25.406000+0000	PortScan
80	2026-07-13 18:19:56.721000+0000	PortScan
80	2026-07-13 18:19:56.659000+0000	PortScan
80	2026-07-13 18:19:56.607000+0000	PortScan

(10 rows)

4. PAPER DRAFT

Abstract

This project proposes a scalable and fault-tolerant real-time data streaming pipeline designed for network intrusion detection. Operating on the extensive CICIDS2017 dataset, the architecture simulates a high-throughput live network environment by artificially injecting temporal attributes and continuously replaying historical flow records. The system orchestrates Apache Kafka for efficient message buffering and partition-based load balancing, leveraging "Destination Port" cardinality to mimic realistic traffic routing. At the core of the pipeline, Apache DataFusion executes low-latency analytical queries and micro-batch windowing functions to detect anomalies and aggregate traffic metrics. Identified threats and computed metrics are persistently stored in an Apache Cassandra distributed database. Evaluated on a containerized local ecosystem via Docker Compose, this research aims to assess the pipeline's overall throughput, query execution latency, and systemic resilience under heavy synthetic network loads.

Related Work

Prior research in streaming network intrusion detection systems has predominantly relied on JVM-heavy frameworks such as Apache Spark Structured Streaming or Apache Flink. Kumar et al. [2] propose a distributed IDS built on Apache Kafka and Apache Spark, using Kafka's streaming platform for reliable, low-latency data delivery combined with Spark MLlib for parallelized classification, while an earlier empirical study [3] specifically benchmarks the performance of Kafka-Spark Streaming pipelines for intrusion-detection transaction processing. Studies utilizing the CICIDS2017 dataset - introduced by Sharafaldin, Lashkari, and Ghorbani [1] as a contemporary, realistic benchmark for intrusion detection research - frequently focus on the accuracy of machine learning models rather than

the underlying data engineering infrastructure required to process millions of records in real time. Recent advancements in Rust-based query engines, notably Apache DataFusion [4], have demonstrated significant improvements in memory efficiency and execution speed relative to JVM-based engines through its Arrow-native, vectorized execution model. However, literature integrating DataFusion with Apache Kafka and NoSQL databases like Cassandra for continuous cybersecurity monitoring remains limited. This project aims to bridge that gap by demonstrating a micro-batching integration strategy for these technologies.

Proposed Implementation

The proposed architecture orchestrates data flow across four distinct operational layers:

1. **Data Ingestion:** Static CSV records from the CICIDS2017 dataset are read by an asynchronous Python producer. Real-time UTC timestamps are dynamically injected into each row to facilitate time-based windowing downstream.
2. **Message Broker (Apache Kafka):** Records are serialized into JSON and pushed to a Kafka topic named network-traffic. To ensure parallel processing and optimal load distribution, messages are partitioned using the Destination Port, mimicking realistic network switch behavior.
3. **Stream Processing Engine (Apache DataFusion):** A consumer script reads from Kafka, batching incoming messages. These micro-batches are loaded into a DataFusion Session Context as DataFrames. SQL queries are executed to perform predicate-based filtering (isolating malicious labels) and continuous tumbling-window aggregations (flow counts, benign/attack counts, duration averages), where each window's boundaries are computed by truncating the Injected_Timestamp field of the records in the current micro-batch to a fixed granularity (currently 1 second), rather than using the processing-time timestamp of the batch itself. Consistent with the Systems Track's focus on infrastructure orchestration rather than detection-algorithm design, the dataset's ground-truth Label field is used as the filtering predicate; this exercises the same predicate-pushdown and partition-routing pathway that a production threshold- or model-based alert would use, without requiring a bespoke detection model to be developed as part of this project.
4. **Distributed Storage (Apache Cassandra):** Analytical outputs are persisted into a Cassandra cluster. `flagged_anomalies` stores event-level data for distinct attacks, while `windowed_traffic_metrics` acts as an aggregated time-series repository suitable for dashboard visualizations.

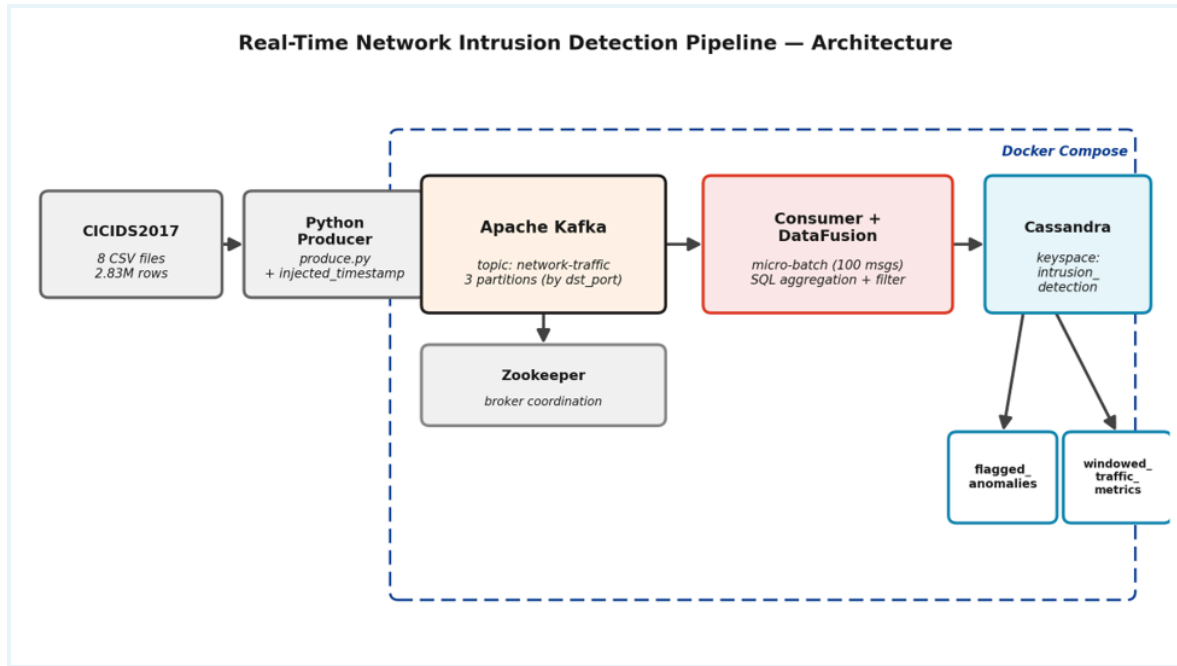


Figure 1. End-to-end pipeline architecture: CSV ingestion, Kafka partitioning, DataFusion micro-batch processing, and Cassandra persistence, orchestrated via Docker Compose.

Evaluation Plan: The system will be evaluated by scaling the ingestion rate (messages per second) to identify the maximum throughput before consumer lag accumulates. Additionally, execution latency for DataFusion micro-batches and Cassandra write throughput will be continuously monitored during the full-scale benchmark.

5. NEXT STEPS

- **Full-Scale Performance Benchmarking:** The producer's `--rate` parameter will be progressively increased (e.g., 100 → 1,000 → 5,000 msgs/sec) against the full 2.8M-record dataset (all eight daily CSV files streamed sequentially through the producer) to identify the maximum sustainable throughput before consumer lag accumulates, rather than replaying the entire dataset at the demo rate of 100 msgs/sec.
- **Fault-Tolerance Testing:** Extend the Docker Compose setup to run Kafka with multiple brokers (replication factor ≥ 2) and Cassandra with multiple nodes (replication factor ≥ 2), then simulate node/broker failure during active ingestion (e.g., docker stop) to measure message loss via producer/consumer offset comparison, consumer lag recovery time via Kafka consumer group lag metrics, and query availability by issuing read queries against the surviving Cassandra node(s) during the outage window.
- **Final Technical Report Generation:** The findings from the scalability and fault-tolerance tests will be compiled into the final academic paper, in accordance with the BIL401 specifications.

6. REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP), Portugal, 2018, pp. 108-116.
- [2] K. M. K. Kumar, M. Sreekar, K. Ullas, and S. Muthuraman, "Distributed Intrusion Detection System using Kafka and Spark Streaming," in Proc. 2025 Int. Conf. Visual Analytics and Data Visualization (ICVADV), 2025.
- [3] M. T. Tun, D. E. Nyaung, and M. P. Phyu, "Performance Evaluation of Intrusion Detection Streaming Transactions Using Apache Kafka and Spark Streaming," in Proc. 2019 Int. Conf. Advanced Information Technologies (ICAIT), 2019, pp. 25–30, doi: 10.1109/AITC.2019.8920960.